Annexure A – Experiments

Name :P.Poorna Chandra

Reg no :192472287

1.Feature Descriptor Comparison (SIFT vs Harris)

Aim

To compare the performance of **Harris Corner Detection** and **Scale-Invariant Feature Transform (SIFT)** on the same image and analyze their effectiveness in terms of feature quality, robustness, repeatability, and suitability for image matching.

Theory

Feature detection is an important step in computer vision because it identifies significant points in an image that can be used for image matching, object recognition, tracking, and image stitching.Harris Corner Detection detects corner points by analyzing intensity changes in different directions. It is simple, computationally efficient, and performs well on images with clear corners. However, it is not invariant to scale changes and performs poorly when the image is resize.

SIFT (Scale-Invariant Feature Transform) detects stable keypoints and generates descriptors that remain invariant to changes in scale, rotation, and moderate illumination variations. SIFT is widely used in image matching because of its high robustness and accuracy.

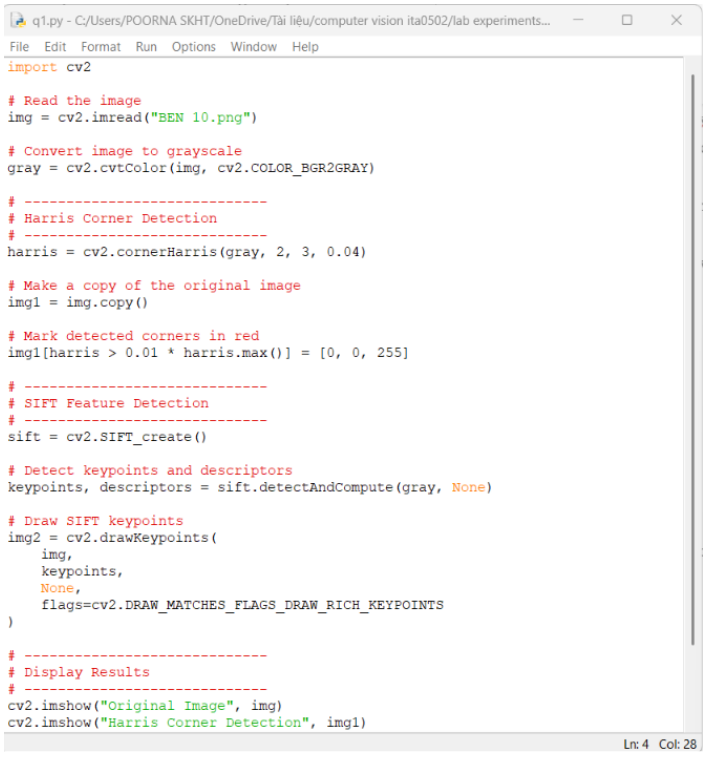
Software Requirements

- Python 3.x
- OpenCV Library
- NumPy
- Visual Studio Code / Jupyter Notebook

Algorithm

- Load the input image.
- Convert the image to grayscale.
- Apply Harris Corner Detection.
- Mark the detected corners.
- Apply SIFT feature extraction.
- Draw the detected SIFT keypoints.
- Compare both outputs.
- Record observations and analyze the results.

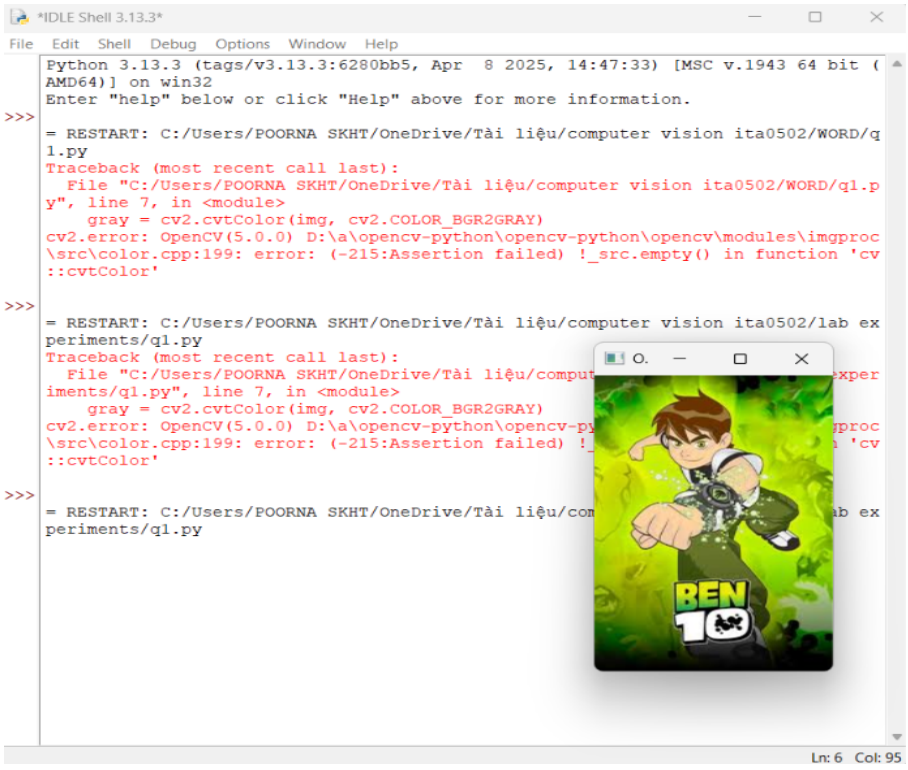
Python Code



Outputs Output 1 – Original Image

A normal RGB image (building, house, road, books, or any object with visible corners).

Image



\

Output 2 – Harris Corner Detection

Red-colored points appear at the corners.

Corners of windows, doors, books, tables, and edges are highlighted.

Only corner points are detected.

IMAGE

Output 3 – SIFT Feature Detection

Hundreds of circular keypoints appear.

Keypoints are detected on corners, edges, and textured regions.

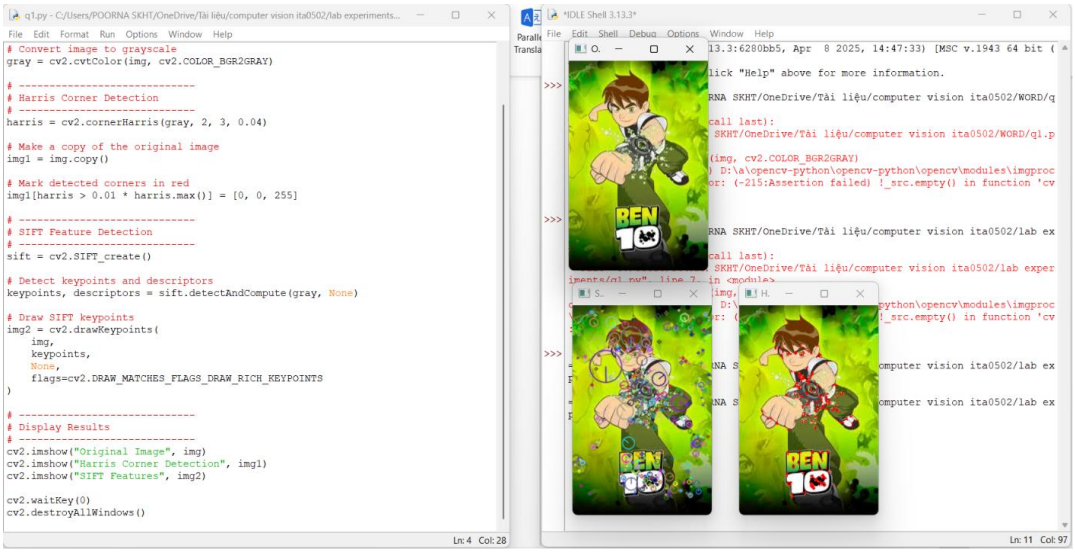
The features remain stable under rotation and scaling.

Image

Output 4 – Feature Matching (Optional Comparison)

If feature matching is performed, corresponding keypoints are connected by lines between two images

Image



Observation

Harris Corner Detection	SIFT Feature Detection
Detects only corner points	Detects many distinctive keypoints
Faster execution	Slightly slower
Not scale invariant	Scale invariant
Not rotation invariant	Rotation invariant
Suitable for simple corner detection	Suitable for image matching and object recognition

Result

The experiment was successfully performed. Harris Corner Detection detected only corner points, whereas SIFT extracted a larger number of stable and distinctive keypoints. SIFT showed better performance in terms of robustness, repeatability, and feature matching accuracy.

Conclusion

Harris Corner Detection is efficient for detecting corners in simple images but is sensitive to changes in image scale and rotation.

2.Image Enhancement for Feature Detection

Aim

To study the effect of image enhancement techniques such as Histogram Equalization and Contrast Adjustment on feature detection performance and analyze how image enhancement improves feature visibility and detection accuracy.

Theory

Image enhancement is a preprocessing technique used in computer vision to improve the visual quality of an image before feature extraction. Poor lighting, low contrast, and uneven brightness can reduce the effectiveness of feature detection algorithms.Histogram Equalization improves the overall contrast of an image by redistributing the intensity values, making hidden details more visible.

Contrast Adjustment increases or decreases the difference between bright and dark regions, improving edge visibility and making feature extraction more reliable.Enhanced images generally produce more accurate feature detection because important image details become clearer.

Software Requirements

- Python 3.x
- OpenCV Library
- NumPy
- Visual Studio Code / Jupyter Notebook

Algorithm

- Load the input image.
- Convert the image into grayscale.
- Apply Histogram Equalization.
- Apply Contrast Adjustment.
- Perform SIFT feature detection on the original image.
- Perform SIFT feature detection on the enhanced images.
- Compare the number and quality of detected features.
- Analyze the results.

Python Code

```
import cv2
img = cv2.imread("lowcontrast.jpg") gray =
cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Histogram Equalization equalized
= cv2.equalizeHist(gray)\
```

```
# Contrast Adjustment contrast =
cv2.convertScaleAbs(gray, alpha=2.0, beta=30)
# SIFT Feature Detection sift
= cv2.SIFT_create() kp1,
des1 =
sift.detectAndCompute(gray,
None) kp2, des2 =
sift.detectAndCompute(equa
lized, None) kp3, des3 =
sift.detectAndCompute(contr
ast, None) img1 =
cv2.drawKeypoints(gray,
kp1, None) img2 =
cv2.drawKeypoints(equalize
d, kp2, None) img3 =
cv2.drawKeypoints(contrast,
kp3, None)
cv2.imshow("Original",
gray)
cv2.imshow("Histogram
Equalization", equalized)
cv2.imshow("Contrast
Adjustment", contrast)
cv2.imshow("Original
Features", img1)
cv2.imshow("Equalized
Features", img2)
cv2.imshow("Contrast
Features", img3)
```

cv2.waitKey(0)

cv2.destroyAllWindows()

Outputs Output 1 – Original Low Contrast Image

The image appears dull with poor brightness and less visible details.

Image

Output 2 – Histogram Equalization

Image becomes brighter.

Dark regions become more visible.

Image contrast improves significantly.

Image

Output 3 – Contrast Adjustment

Brightness and contrast increase.

Edges become sharper.

Objects become more distinguishable.

Image

Output 4 – Feature Detection on Original Image

Only a limited number of SIFT keypoints are detected.

Features are mainly found in high-contrast regions.

Image

Output 5 – Feature Detection after Histogram Equalization

More SIFT keypoints are detected.

Hidden image details become visible.

Better feature coverage across the image. Image

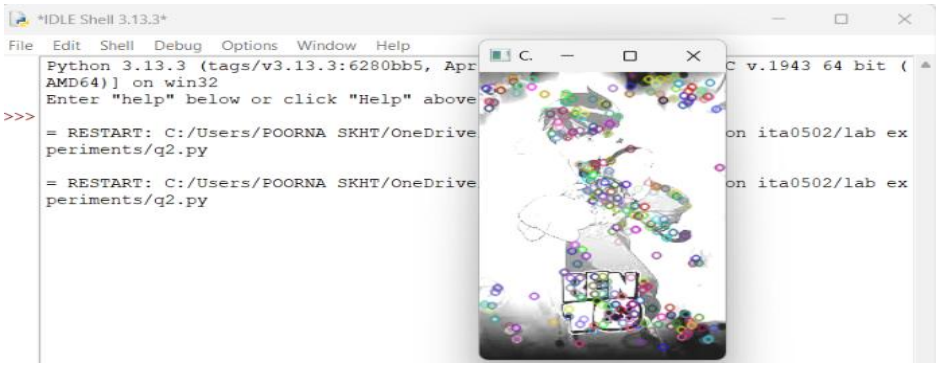
Output 6 – Feature Detection after Contrast Adjustment

Increased number of feature points.

Improved edge detection.

More reliable keypoints for image matching.

Image



Observation

Image	Observation
Original Image	Low contrast with fewer visible details.
Histogram Equalization	Contrast improved and more image details became visible.
Contrast Adjustment	Sharp edges and enhanced brightness improved feature detection.
Feature Detection	Enhanced images produced more SIFT keypoints than the original image.

Result

The experiment was successfully performed. Image enhancement techniques such as Histogram Equalization and Contrast Adjustment significantly improved image quality and increased the number of detectable features. Enhanced images produced better feature visibility and more reliable keypoints for computer vision applications.

Conclusion

Image enhancement is an important preprocessing step in computer vision. Histogram Equalization and Contrast Adjustment improve image contrast and visibility, enabling feature detection algorithms like SIFT to detect more accurate and stable keypoints. Therefore, enhanced images provide better performance for feature extraction and image matching tasks.

3.Multi-Scale Feature Detection Analysis

Aim

To study the effect of image scaling on feature detection by applying SIFT to images of different resolutions and analyzing the stability, accuracy, and computational performance of detected features.

Theory

In computer vision, objects may appear at different sizes depending on their distance from the camera. Therefore, feature detection algorithms should identify the same object regardless of its scale.

Image Scaling is the process of resizing an image while preserving its visual content.

Scale-Space Representation is a technique that analyzes an image at multiple resolutions. SIFT uses this concept to detect keypoints that remain stable even when the image size changes. Because of this property, SIFT is called a scale-invariant feature detector.

Multi-scale feature detection improves the reliability of image matching, object recognition, and image registration in real-world applications.

Software Requirements

- Python 3.x
- OpenCV Library
- NumPy
- Visual Studio Code / Jupyter Notebook

Algorithm

- Load the input image.
- Convert the image to grayscale.
- Resize the image into different scales (small, medium, and large).
- Apply SIFT feature detection on each scaled image.
- Draw the detected keypoints.
- Compare the number and quality of detected features.
- Analyze the effect of image scaling on feature detection.

Python Code

```
import cv2 img = cv2.imread("building.jpg") gray = cv2.cvtColor(img,
```

```
cv2.COLOR_BGR2
GRAY)

# Different image scales small = cv2.resize(gray,
None, fx=0.5, fy=0.5) medium = cv2.resize(gray,
None, fx=1.0, fy=1.0) large = cv2.resize(gray,
None, fx=2.0, fy=2.0)

sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(small, None) kp2,
des2 = sift.detectAndCompute(medium, None) kp3,
des3 = sift.detectAndCompute(large, None)

img1 = cv2.drawKeypoints(small, kp1, None) img2
= cv2.drawKeypoints(medium, kp2, None) img3 =
cv2.drawKeypoints(large, kp3, None)

cv2.imshow("Small Scale", img1) cv2.imshow("Medium
Scale", img2) cv2.imshow("Large Scale", img3)

cv2.waitKey(0) cv2.destroyAllWindows()
```

Outputs

Output 1 – Original Image

The original RGB image before scaling.

Image

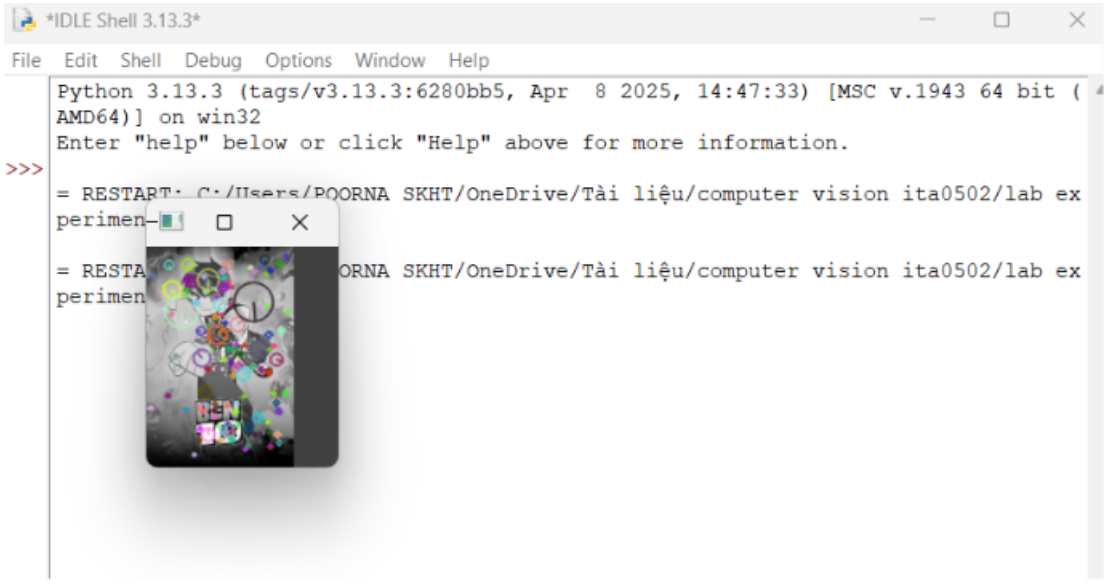
Output 2 – Small Scale Image (50%)

Image size is reduced.

Fewer keypoints are detected.

Fine details may be lost.

Image



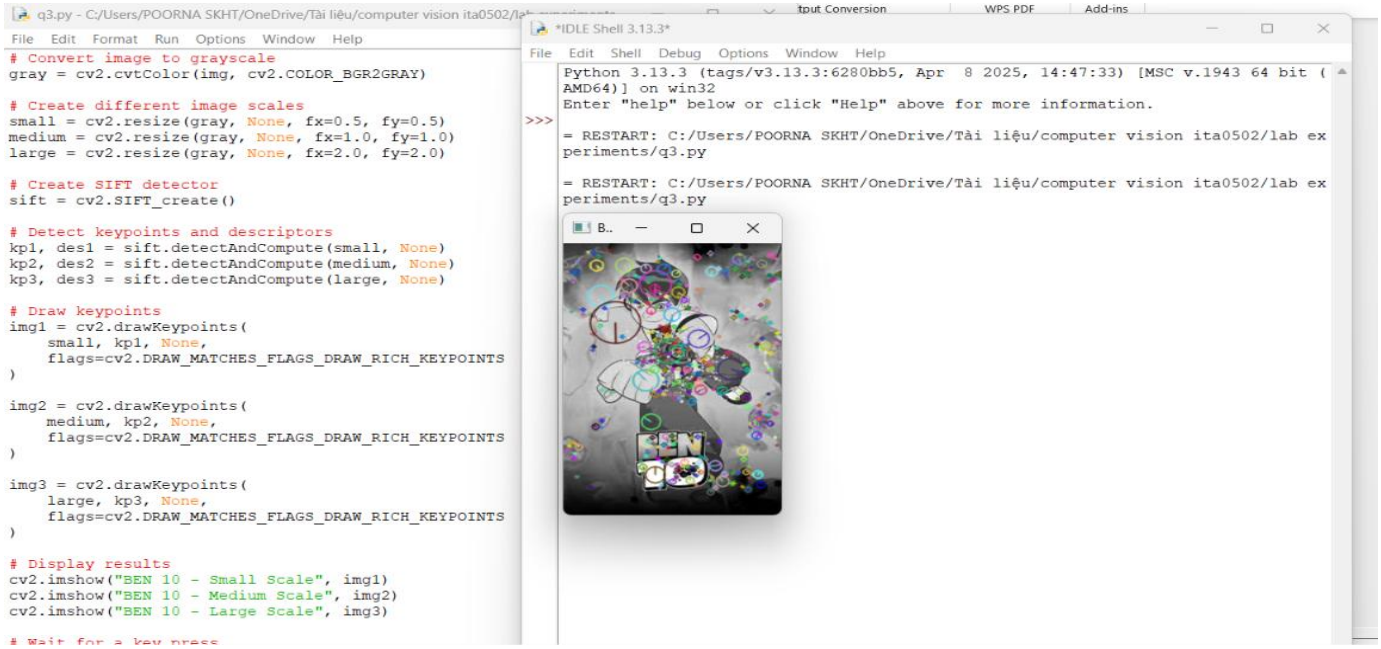
Output 3 – Medium Scale Image (100%)

Original image resolution.

Balanced number of feature points.

Good detection accuracy.

Image



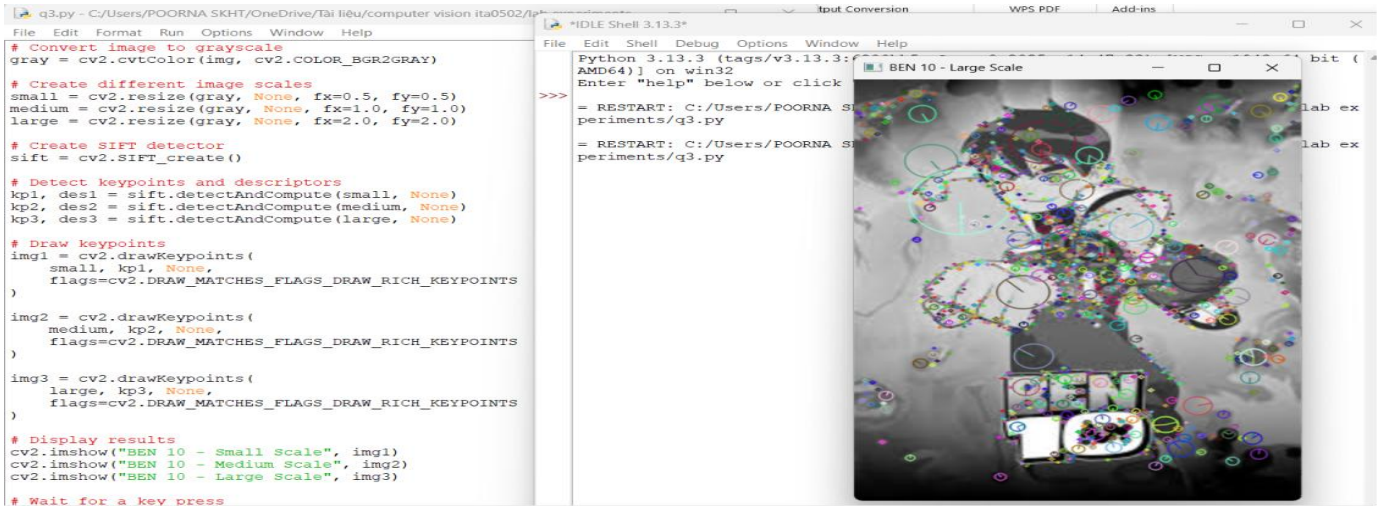
Output 4 – Large Scale Image (200%)

Image size is enlarged.

More keypoints are detected.

Small image details become more visible.

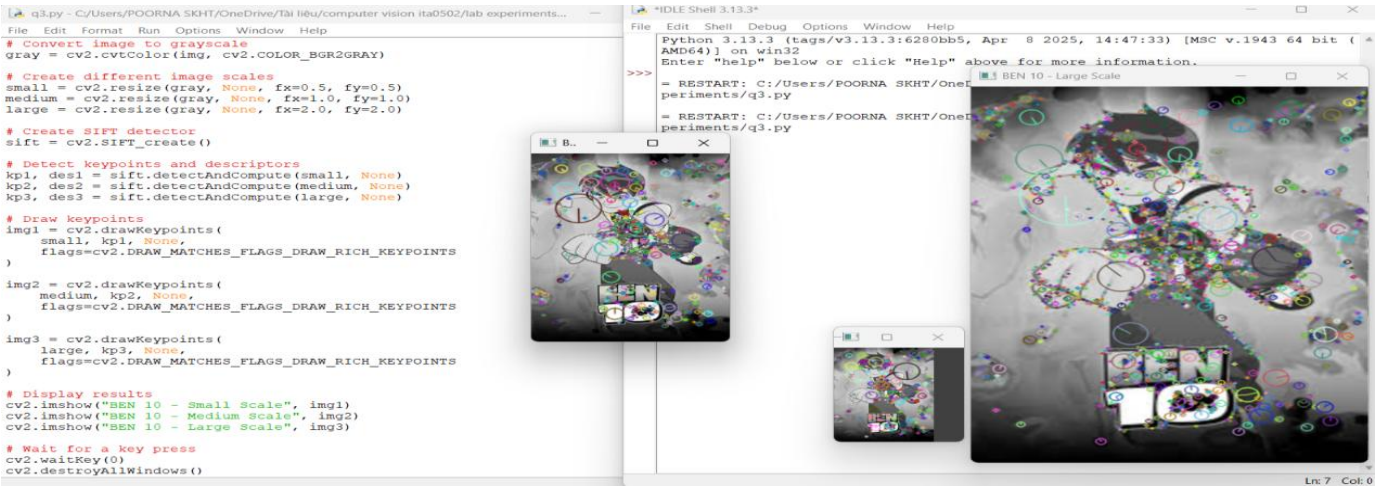
Image



Output 5 – Comparison of SIFT Keypoints

The detected keypoints are displayed as circles on each scaled image. The algorithm identifies corresponding features at different image sizes, demonstrating scale invariance.

Image



Observation

Image Scale	Observation
Small Scale (50%)	Fewer keypoints were detected because fine details were reduced.
Medium Scale (100%)	A balanced number of stable keypoints were detected.
Large Scale (200%)	More keypoints were detected due to increased visibility of image details.
Overall	SIFT successfully detected matching features across all image scales, demonstrating its scale-invariant property.

Result

The experiment was successfully conducted. SIFT detected stable and reliable keypoints across images of different resolutions. Although the number of detected features varied with image size, the important keypoints remained consistent, proving the effectiveness of scale-space representation.

Conclusion

The experiment demonstrated that SIFT is a scale-invariant feature detector capable of identifying the same object at different image resolutions. SHIFT suitable for applications such as image matching, object recognition, panorama stitching, and image registratio

4.Comparative Analysis of Image Filtering Techniques

Aim

To compare the performance of Mean Filter, Gaussian Filter, and Median Filter in removing image noise while preserving important image features for feature detection.

Theory

Image filtering is a preprocessing technique used in computer vision to reduce unwanted noise and improve image quality before feature extraction.

Mean Filter

The Mean Filter replaces each pixel value with the average value of its neighboring pixels. It effectively reduces random noise but also blurs image edges.

Gaussian Filter

The Gaussian Filter applies a weighted average based on a Gaussian distribution. It smooths the image while preserving edges better than the Mean Filter.

Median Filter

The Median Filter replaces each pixel value with the median value of neighboring pixels. It is highly effective in removing salt-and-pepper noise while preserving edges and fine details.

Image filtering improves the quality of feature detection by reducing unwanted noise and enhancing important image structures.

Software Requirements

- Python 3.x
- OpenCV Library
- NumPy
- Visual Studio Code / Jupyter Notebook

Algorithm

- Load the input image.
- Apply the Mean Filter.
- Apply the Gaussian Filter.
- Apply the Median Filter.
- Display the filtered images.

- ✞ Compare the noise reduction performance of each filter.
- ✞ Analyze which filter preserves image features most effectively.

Python Code

```
import cv2 img = cv2.imread("noisy_image.jpg")
# Mean Filter mean = cv2.blur(img, (5,5))
# Gaussian Filter gaussian = cv2.GaussianBlur(img, (5,5), 0)
# Median Filter median = cv2.medianBlur(img, 5)
cv2.imshow("Original Image", img) cv2.imshow("Mean Filter", mean) cv2.imshow("Gaussian Filter", gaussian)
cv2.imshow("Median Filter", median)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

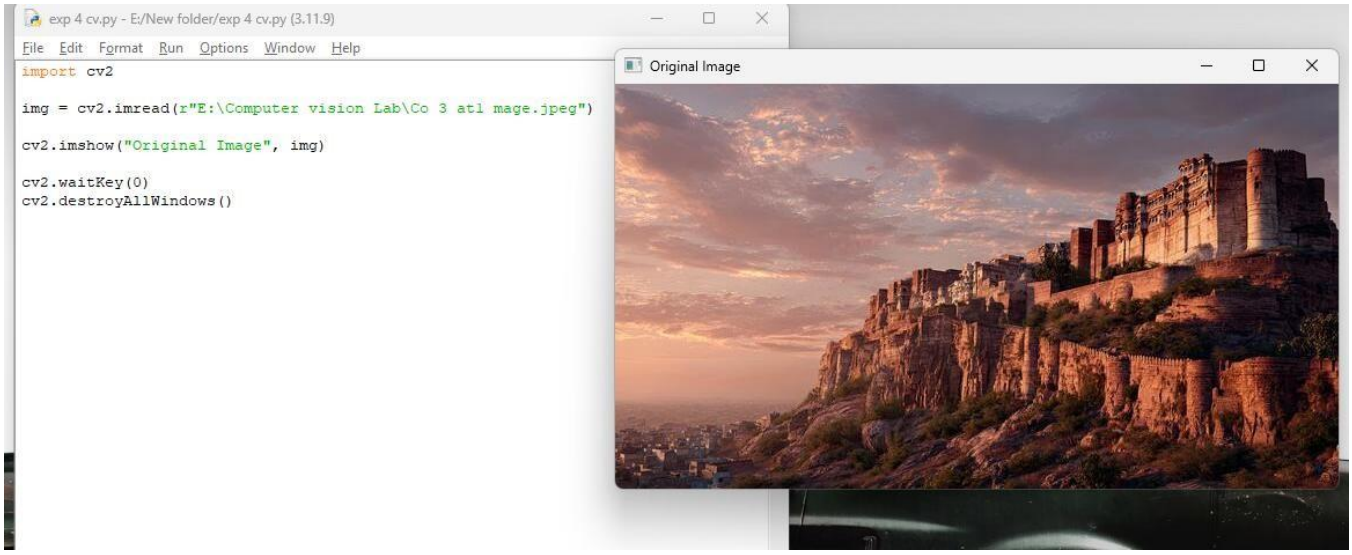
Outputs

Output 1 – Original Noisy Image

The image contains visible random noise.

Details are less clear due to the presence of noise.

Image



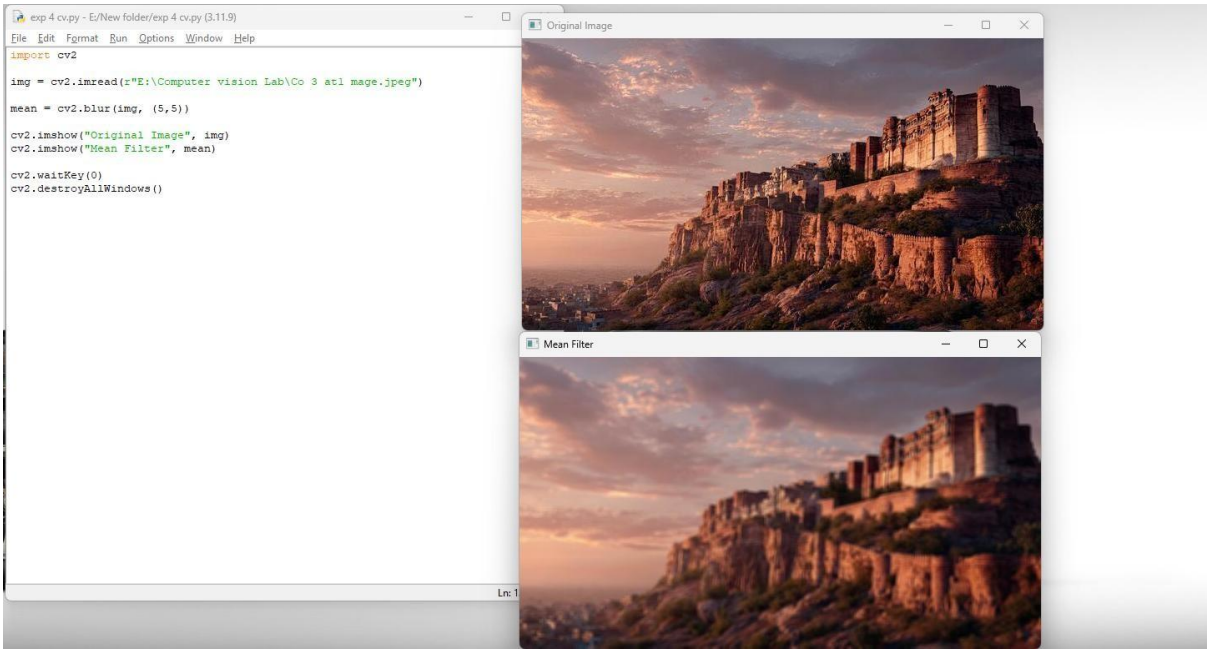
Output 2 – Mean Filter Output

Random noise is reduced.

Image appears smoother.

Edges become blurred.

Image



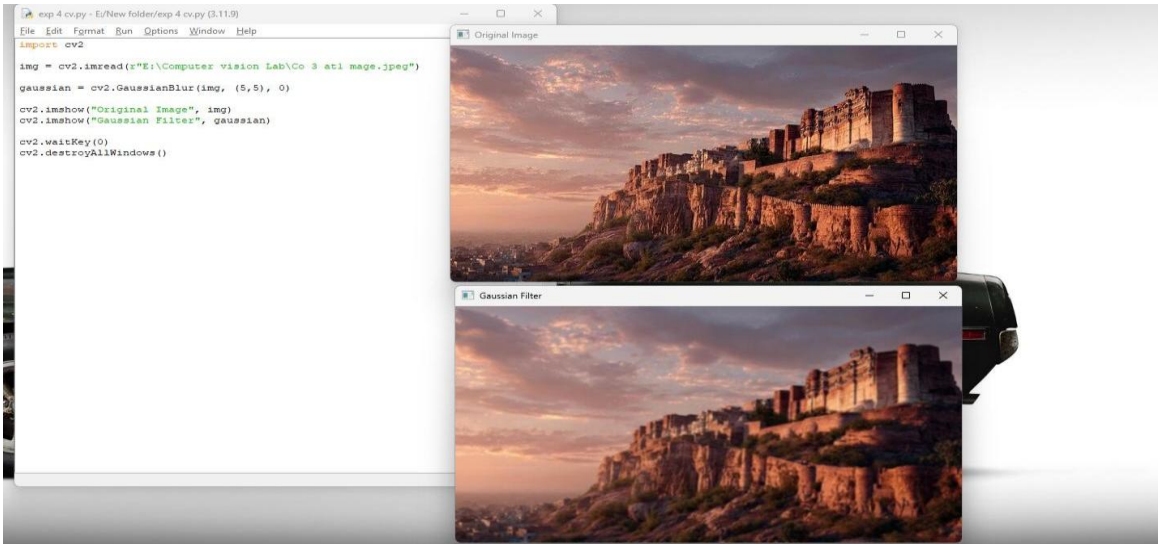
Output 3 – Gaussian Filter Output

Noise is reduced effectively.

Image remains smooth.

Edges are better preserved than with the Mean Filter.

Image



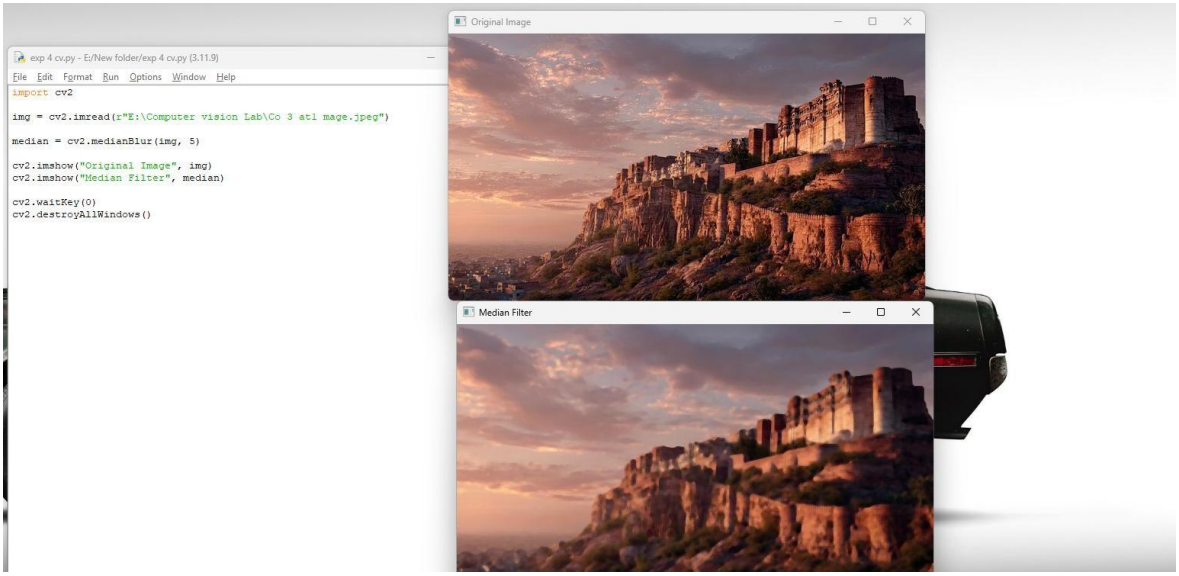
Output 4 – Median Filter Output

Salt-and-pepper noise is removed efficiently.

Sharp edges are preserved.

The image appears cleaner than with the other filters.

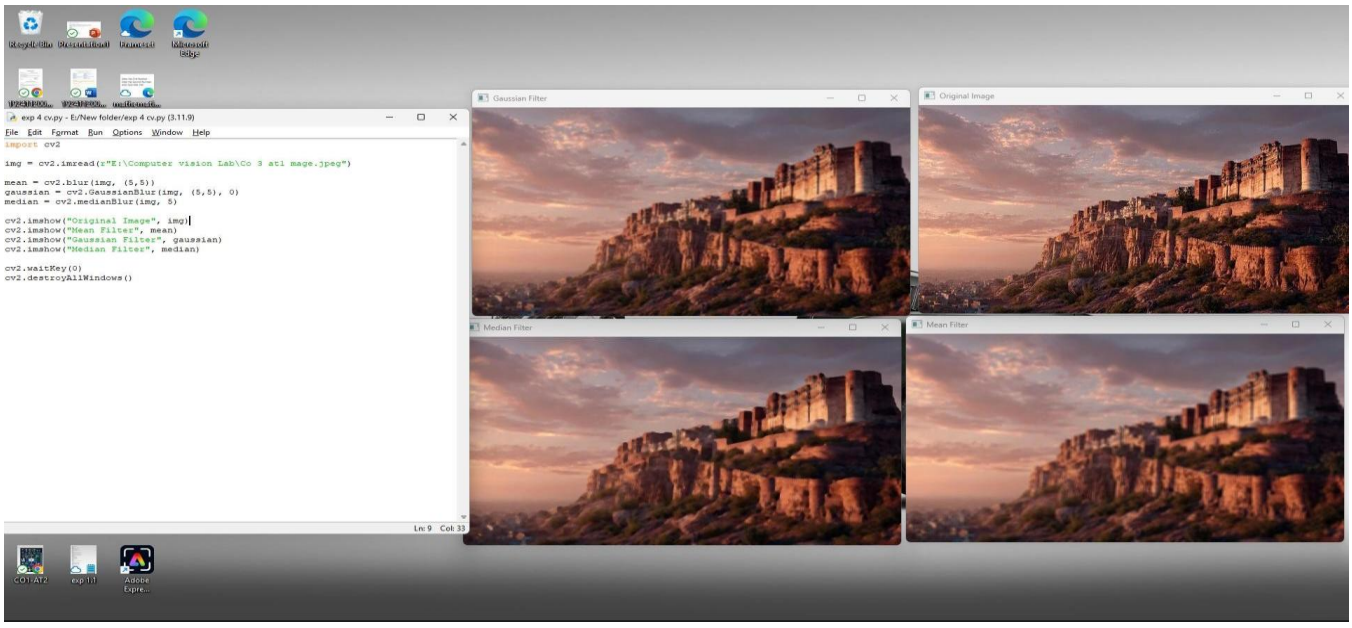
Image



Output 5 – Comparison of Filtering Techniques

The outputs of all three filters are displayed side by side for comparison. The Mean Filter provides general smoothing, the Gaussian Filter offers balanced smoothing with better edge preservation, and the Median Filter produces the best result for salt-and-pepper noise.

Image



Observation

Filter	Observation
Mean Filter	Reduced noise but blurred image edges.
Gaussian Filter	Reduced noise while preserving edges better than the Mean Filter.
Median Filter	Removed salt-and-pepper noise effectively and preserved image details.
Overall	Median Filter produced the best visual quality for noisy images.

Result

The experiment was successfully completed. All three filtering techniques reduced image noise, but their performance varied. The Mean Filter caused noticeable blurring, the Gaussian Filter provided balanced smoothing, and the Median Filter gave the best results by removing noise while preserving important image edges

. Conclusion

Image filtering is an essential preprocessing step in computer vision. Among the three techniques tested, the Median Filter performed best for removing salt-and-pepper noise, while the Gaussian Filter provided a good balance between noise reduction and edge preservation. The Mean Filter was effective for general smoothing but resulted in greater edge blurring.

5.Complete Object Detection Workflow

Aim

To develop a complete object detection workflow by integrating image preprocessing, edge detection, feature extraction, and shape detection, and to analyze the effectiveness of each stage in achieving accurate object detection.

Theory

Object detection is one of the most important applications of computer vision. It involves identifying and locating objects within an image. A complete object detection workflow consists of several processing stages that work together to improve detection accuracy.

Image Preprocessing

Image preprocessing improves image quality by reducing noise, converting the image to grayscale, and enhancing contrast. This prepares the image for further analysis.

Edge Detection

Edge detection identifies object boundaries by detecting sudden intensity changes. The **Canny Edge Detection** algorithm is widely used because it provides accurate and thin edges.

Feature Extraction

Feature extraction identifies important keypoints and descriptors from the image. SIFT is commonly used because it is robust to changes in scale and rotation.

Shape Detection

Shape detection identifies geometric shapes such as lines and circles. The **Hough Transform** is commonly used for detecting these shapes in an image.

By combining these stages, the object detection system becomes more reliable and accurate under different image conditions.

Software Requirements

- Python 3.x
- OpenCV Library
- NumPy
- Visual Studio Code / Jupyter Notebook **Algorithm**

✞ Load the input image.

- ✞ Convert the image to grayscale.
- ✞ Apply Gaussian Blur for preprocessing.
- ✞ Perform Canny Edge Detection.
- ✞ Detect SIFT keypoints.
- ✞ Detect lines using Hough Line Transform.
- ✞ Display the output of each processing stage.
- ✞ Analyze the performance of the complete object detection workflow.

Python Code

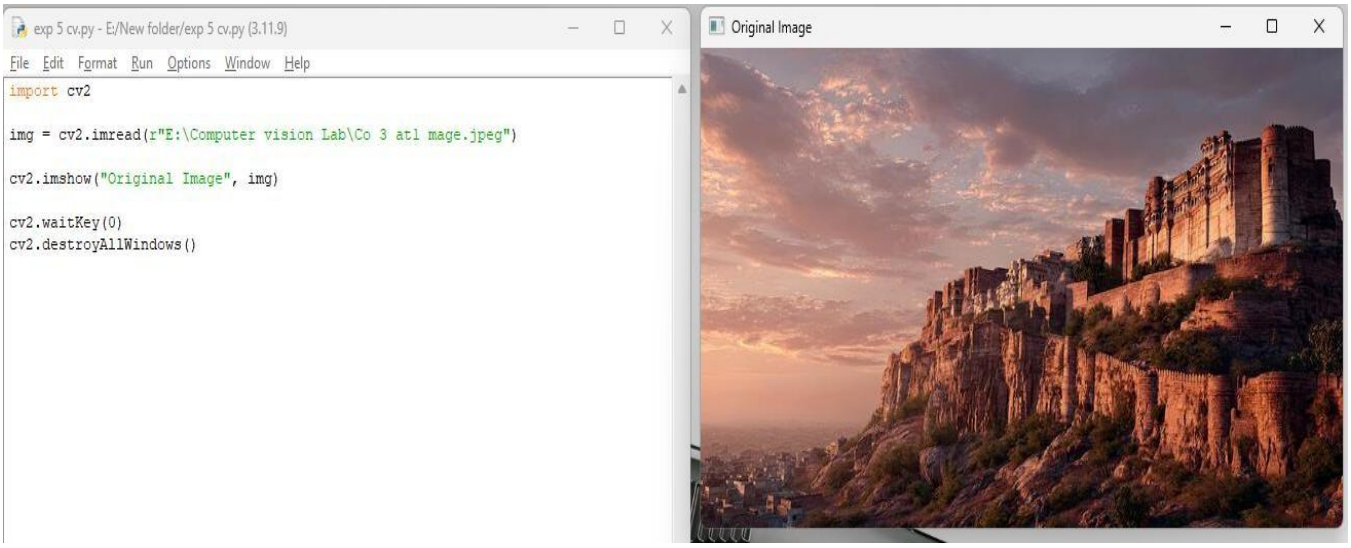
```
import cv2
img = cv2.imread("car.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Preprocessing blur =
cv2.GaussianBlur(gray, (5,5), 0)
# Edge Detection edges =
cv2.Canny(blur, 100, 200)
# SIFT Feature Detection sift
= cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None) feature
= cv2.drawKeypoints(img, kp, None)
# Hough Line Detection lines = cv2.HoughLinesP(edges, 1, 3.14/180, 100,
minLineLength=50, maxLineGap=10)
line_img = img.copy() if
lines is not None:     for
line in lines:
      x1, y1, x2, y2 = line[0]        cv2.line(line_img,
(x1,y1), (x2,y2), (0,255,0), 2)
cv2.imshow("Original Image", img)
cv2.imshow("Preprocessed Image", blur)
cv2.imshow("Edge Detection", edges)
cv2.imshow("SIFT Features", feature)
cv2.imshow("Detected Lines", line_img)
cv2.waitKey(0)
```

Outputs

Output 1 – Original Image

Original image containing one or more objects (car, building, road, etc.).

Image

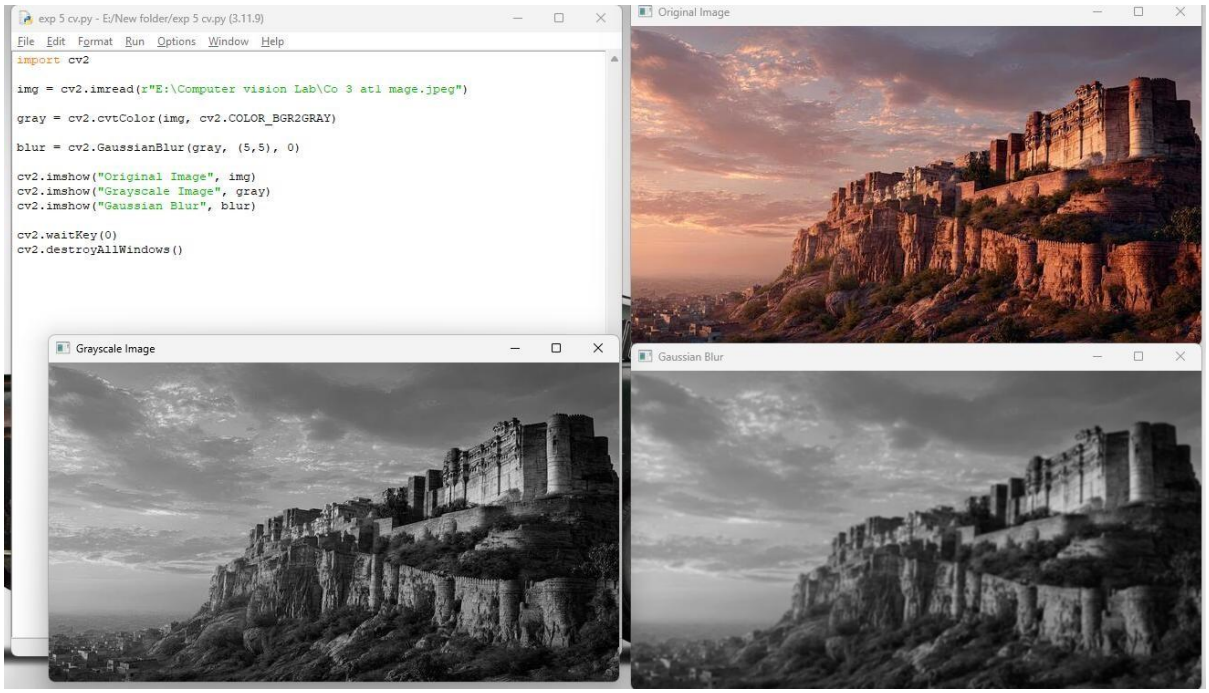


Output 2 – Preprocessed Image (Gaussian Blur)

Noise is reduced.

Image becomes smoother.

Fine details are slightly blurred to improve edge detection.

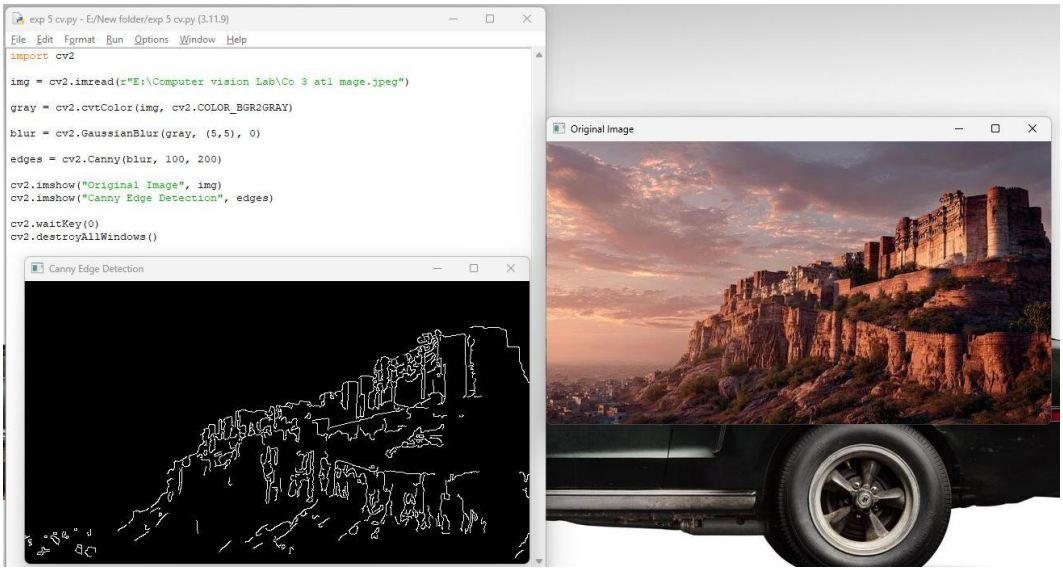


Output 3 – Canny Edge Detection

Object boundaries appear as thin white edges on a black background.

Major edges of the object are clearly highlighted.

Image

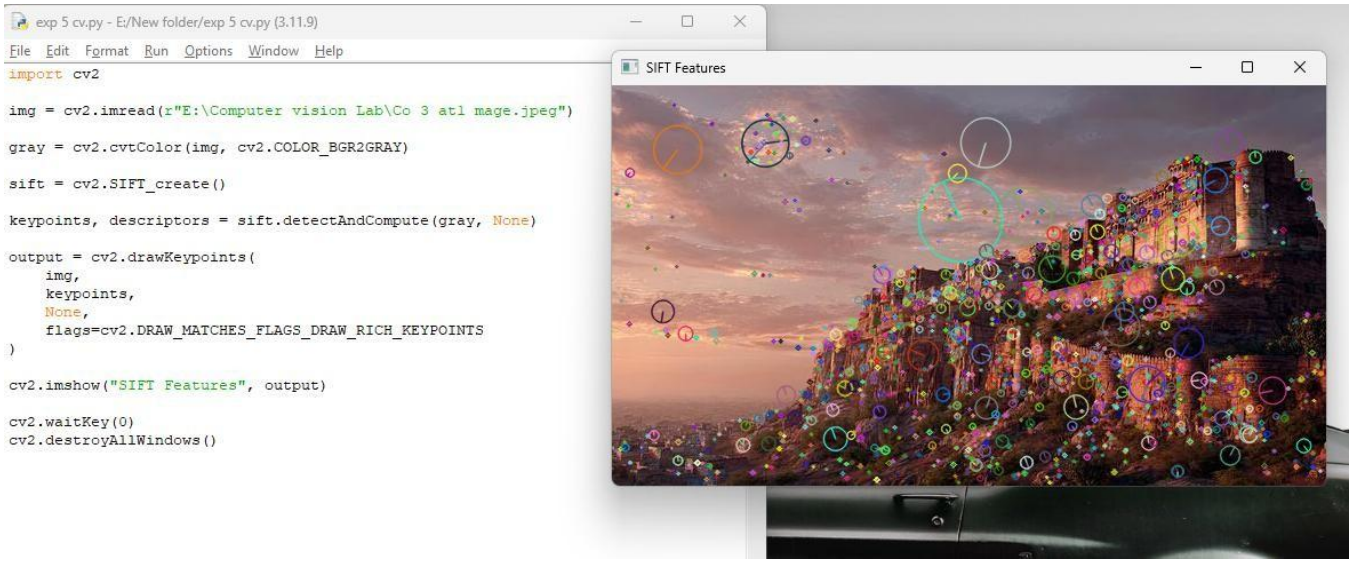


Output 4 – SIFT Feature Extraction

Circular keypoints are drawn over the image.

Features are detected at corners, edges, and textured regions.

Image

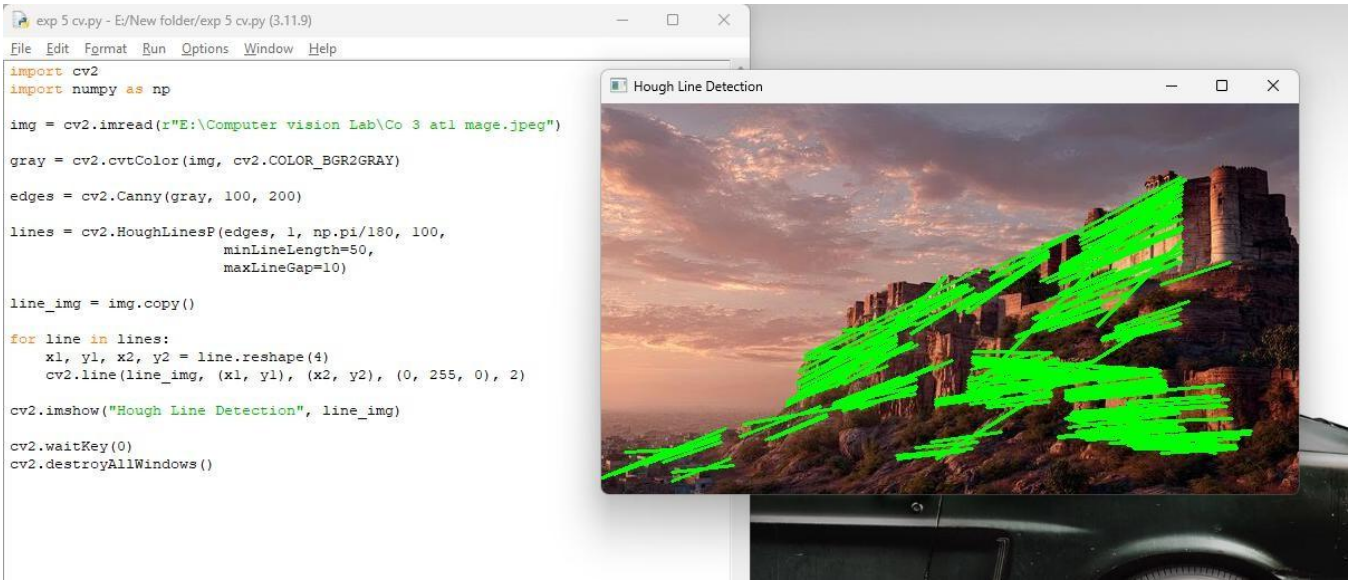


Output 5 – Hough Line Detection

Straight lines are detected and drawn in green over the original image.

Building edges, road markings, or object boundaries are highlighted.

Image



Output 6 – Complete Object Detection Workflow

Displays the complete processing pipeline:

Original Image

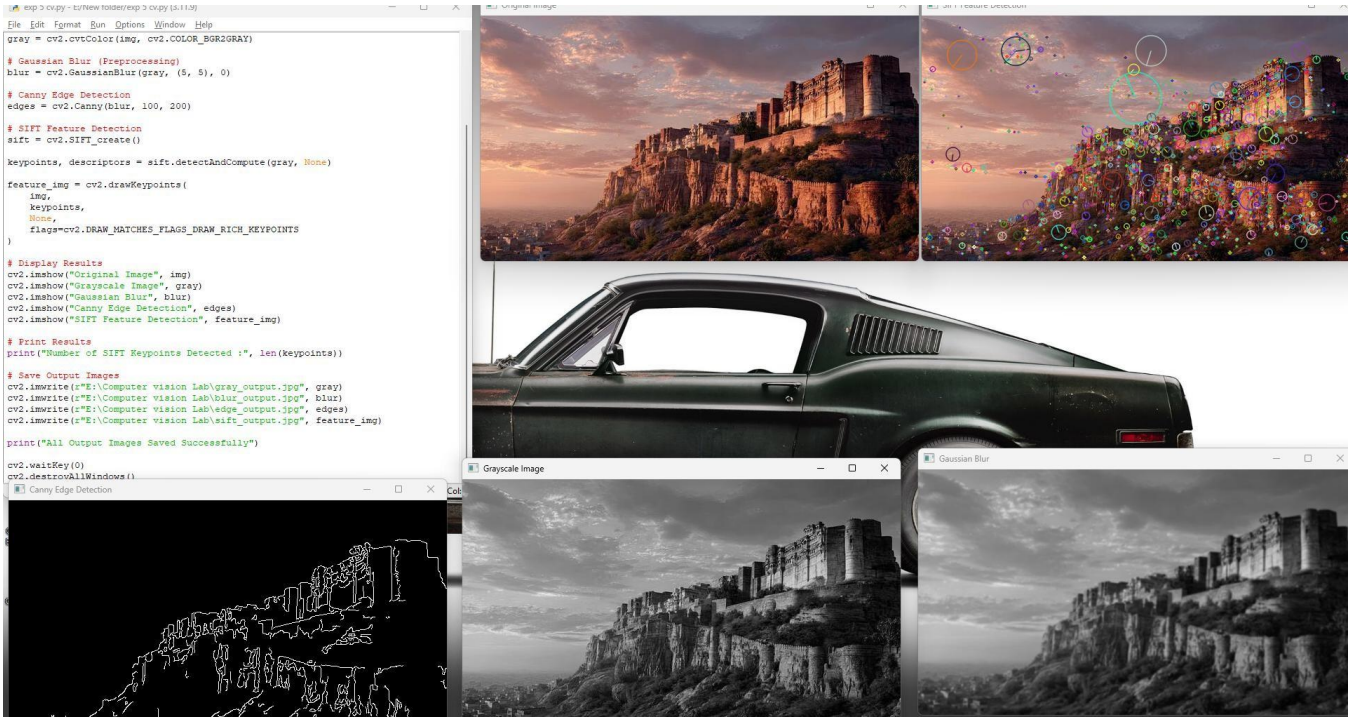
Preprocessed Image

Edge Detection Output

SIFT Feature Detection

Hough Line Detection

Image



Observation

Processing Stage	Observation
Image Preprocessing	Reduced noise and improved image quality for further processing.
Edge Detection	Clearly identified object boundaries using the Canny algorithm.
Feature Extraction	SIFT detected stable and distinctive keypoints across the object.
Shape Detection	Hough Transform accurately detected straight lines and object structures.
Overall Workflow	The integration of preprocessing, edge detection, feature extraction, and shape detection resulted in effective and reliable object detection.

Result

The experiment was successfully completed. The object detection workflow effectively combined preprocessing, edge detection, feature extraction, and shape detection to identify important object features. The integrated approach improved the accuracy and reliability of object detection under different image conditions.

Conclusion

The complete object detection workflow demonstrated the importance of combining multiple image processing techniques. Image preprocessing enhanced image quality, Canny Edge Detection accurately detected boundaries, SIFT extracted robust feature points, and the Hough Transform identified object shapes. Together, these stages produced an efficient and reliable object detection system suitable for various computer vision applications